2024 7th International Conference on Advanced Communication Technologies and Networking (CommNet) | 979-8-3503-6702-7/24/$31.00 ©2024 IEEE | DOI: 10.1109/CommNet63022.2024.10793304

Android Malware Detection Using Machine Learning and Neural Network: A Hybrid Approach with Federated Learning

1st Bishal Shyam Purkayastha
Department of Computer Science
University of Staffordshire
London, England
p033599l@student.staffs.ac.uk

2nd Md. Musfiqur Rahman
Department of Computer Science and Engineering
Chittagong University of Engineering and Technology
Chittagong, Bangladesh
u1304005@student.cuet.ac.bd

3rd Dr. Maryam Shahpasand
Department of Computer Science
University of Staffordshire
London, England
maryam.shahpasand@staffs.ac.uk

***Abstract*—Android is the most popular mobile operating system, which makes it a major target for malware attacks. This paper focuses on the growing problem of malware targeting Android devices and presents a new approach for detecting such threats. The proposed method combines machine learning, neural network and Federated Learning to improve malware detection. A majority voting technique is used to select the most important features from the dataset, which helps identify harmful behaviors more effectively. Various machine learning algorithms, including Random Forest and XGBoost, and neural network architectures, such as CNN-LSTM, are evaluated on an Android malware dataset. Federated Learning ensures data privacy during training, addressing key cybersecurity concerns. The study emphasizes the importance of advanced techniques for adaptive malware detection and provides a detailed analysis of performance across multiple models. Results indicate high accuracy, underscoring the efficacy of the proposed methods in combating Android-specific malware.**

***Index Terms*—Malware Detection, Machine Learning, Dimension Reduction, Hybrid Models, Federated Learning**

I. Introduction

Malware is intentionally designed malicious software that disrupts computers, servers, clients, or computer networks. It is also used to leak private information, access systems unauthorizedly, deny access to information, disrupt the user's computer security and privacy, etc [1]. Researchers categorize a numerous number of types of malware, adware, spying software, viruses, worms, AD, rootkit, backdoors, ransomware, rogue software, wipers, and keyloggers [2]. The relevance of Android technology to malware stems from its vast user base and open-source nature, making it a prime target for cybercriminals. Android's ecosystem allows for the installation of applications from diverse sources, including unregulated third-party app stores, creating vulnerabilities for malware infiltration [3]. According to reports, malware targeting Android has grown significantly, with SonicWall identifying over 270,000 malware variants in early 2022. Trend Micro highlighted a surge in malware incidents, forecasting millions of cases by mid-2020. This proliferation poses significant risks to users and organizations, underlining the need for robust detection mechanisms [4].

Roughly 1.2 billion malware and potentially unwanted applications (PUA) currently exist, with 2021 marking the most active year for new malware variants, during which approximately 150 million new programs were identified [4]. As the volume of malware grows, traditional antivirus tools struggle to provide adequate protection, leaving millions of systems vulnerable to hacking. Malware detection has become increasingly challenging due to the easy availability of resources online that teach attackers how to create malware [5]. Additionally, attackers continuously update their malware to newer versions, making detection methods outdated within a short time frame. This rapid evolution underscores the need for robust and adaptive malware detection systems, as a single attack can cause significant organizational data breaches and financial losses [6]. Machine learning (ML) has emerged as a cornerstone in developing advanced malware detection systems, surpassing the limitations of traditional signature-based methods. ML, a subset of artificial intelligence (AI), involves training algorithms on datasets to automatically recognize patterns and make predictions. By learning from large datasets containing both benign and malicious programs, machine learning models can identify unknown threats that lack predefined detection heuristics [7].

Previous research has explored a wide range of ML and deep learning techniques for malware detection, including Support Vector Machines (SVM), K-Nearest Neighbor (KNN), Genetic Algorithms, Bayesian Estimation, AdaBoost, Decision Tree (DT), Random Forest (RF), Logistic Regression (LR), Naïve Bayes (NB), Multi-Layer Perceptron (MLP), Extreme Gradient Boosting (XGB), Convolutional Neural Networks (CNN), Recurrent Neural Networks (RNN), and Long Short-Term Memory (LSTM). These methods have demonstrated varying levels of success in malware detection tasks [7].

In this study, we propose a hybrid approach combining majority voting for feature selection with advanced machine learning and deep learning architectures, including Federated Learning. Federated Learning is a decentralized training technique that allows models to learn from distributed data while ensuring data privacy. This technique is particularly

valuable in cybersecurity, where sensitive information must be safeguarded. The hybrid approach not only leverages the strengths of multiple machine learning algorithms but also ensures better adaptability to evolving malware threats.

II. Related Work

The field of malware detection systems is not a new area of study. Different machine-learning approaches have already been introduced in this field. Hadiprakoso et al. [6] used two different types of datasets in their research for static and dynamic analysis. They used the SMOTE technique to balance the classes and 10- Fold cross-validation to train and validate different machine learning algorithms on different folds of the data including SVM, DT, RF, NB, KNN, MLP, XGBoost, and tested the models again on the collected samples. They also introduced Principle Component Analysis (PCA) to find out the top 10 features. The maximum accuracy was found from the XGB Classifier. Firdausi et al. [8] collected 470 unique malware and benign software samples and employed five classifier algorithms including KNN, NB, SVM, J48 Decision Tree, and MLP. The best performance was achieved by J48 with an accuracy of 96.8 %. Shahadat et al. [5] specified the important features through Random Forest and evaluated various models including KNN, SVM, NB, DT, RF, Logistic Regression (LR), and Hard Voting (HV) using 15-fold cross-validation and get higher accuracy by DT and RF. In another study, Yazdinejad et al. [9] extracted 500 malware and 200 benign records and applied LSTM architecture to build a malware detection system using 10-fold cross-validation where the detection accuracy was found to be 98%. Mahindru et al. [10] designed a new dataset using publicly available data and evaluated various traditional ML algorithms including NB, DT, RF, Simple Logistic (SL), and K-star where SL performs marginally better than others. However, Kumar et al. [11] focused on improving the detection accuracy rate where DT, RF, GB, SVM, LR, XGBoost, and AdaBoost classifiers were trained and validated on a benchmark dataset using cross-validation where they achieved 99.7% accuracy from RF. Rana et al. [12] also focused on supervised machine learning algorithms to identify Android malware. KNN showed the highest performance with an accuracy of 96% and the neural network obtained an average result of 89%. Ban et al. [13] used CNN in the process of Android malware detection. They used a malware dataset containing 28,179 records of the most frequent malware-related incidents that occurred between years 2018 and 2020. The results turn out to be an F1-score of 0.82, with their method at an accuracy of 98%. Wrapping feature selection was proposed by Smmarwar et al. [14] They trained their classifier algorithms including RF, DT, and SVM on the selected features found from the CICInvesAndMal2019 malware dataset and achieved 91.32%, 91.8%, and 82.33% accuracy. In previous studies, researchers had introduced with some feature selection methods that were used separately, different traditional algorithms, some hybrid models like voting classifiers, and neural networks like LSTM, CNN, ANN, RNN, etc. Our aim in this study is to introduce a majority voting technique to select more robust features from a malware dataset and applied different hybrid architectures including stacking classifiers, Variation of Voting Classifiers, CNN-LSTM, and Federated Learning concerning data privacy.

III. Material and Methodology

A. Dataset

The dataset we have used here was built from the network traffic of a virtual computer running on the Unix/Linux platform. Samples of malware for Android devices, innocuous in behavior, were included in this dataset. The dataset used in this study was sourced from Kaggle [15], comprising 100,000 records evenly distributed across two classes: malicious and benign applications, ensuring a balanced dataset. Each record includes 35 features, capturing static and dynamic attributes of the applications. These features were processed through feature selection methods to identify the most significant attributes for malware detection. The dataset was split into 75 :25. seventy-five percent for training and twenty five percent for testing. A random state of 42 was set to ensure the reproducibility of results. (Fig. 1 represents our proposed framework)

B. Data Preprocessing

Data preprocessing ensures the dataset is ready for analysis. For this study, no missing or duplicate values were found. Outliers in the 'State' feature were identified but not removed, as they could provide valuable information. Categorical data were label-encoded, and the dataset was standardized using the StandardScaler method to ensure consistent scaling. Extreme levels of Outliers also affect the model performance. Machine learning models like Decision Tree, and Random Forest are more robust to the outliers. In the dataset, only the State feature has some outliers that can be ignored.

Label Encoding : Machine Learning models perform with mathematical equations that require numerical values. Hence we have to convert the categorical values into numerical values. Since the hash and classification feature is in categorical format, we converted the classification feature into numerical before passing it to the models and dropping the hash column.

Data Standardization : Data Standardization scales the data to a fixed range. It helps to convert the data into a suitable format for algorithms to improve performance. Different Standardization techniques are available in Machine learning domain including StandardScaler, Min-Max Scaling, RobustScaler, MaxAbsScaler etc. We used StandardScaler in our work which helps to scale the data to a mean of 0 and standard deviation of 1.

$$X' = \frac{X - \mu}{\sigma} \tag{1}$$

where X is the original value, μ is the mean of the feature, and σ is the standard deviation of the feature.

Class imbalance can reduce the performance of a machine learning model where the model can be biased towards the majority class. In our case, both the malware and benign class has equal number of samples.

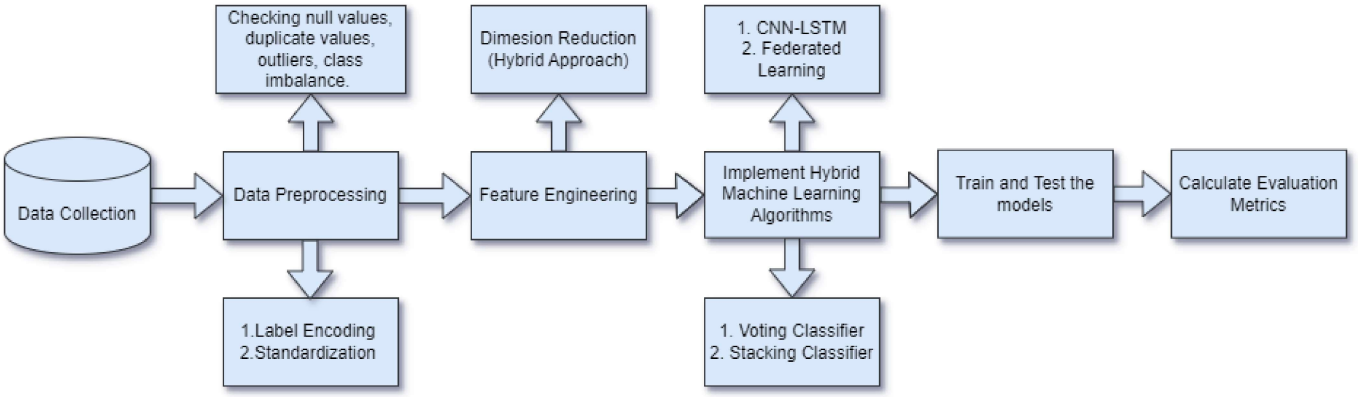

Fig. 1. Our proposed framework

C. Feature Engineering

From Feature Engineering We have used dimension reduction. Dimension reduction selects the most important features only and helps to reduce the computational time, less use of memory, mitigate the curse of dimensionality, improve the model performance, and enhance interpretability. Among different methods, we have used information gain, chi-square test, variance threshold (VT), recursive feature elimination (RFE), and mutual information.

- **Information Gain :** This method examines the relevance of every feature concerning the target variable based on the information gained.
- **Chi-Square Test :** This is a test for statistical significance that measures the independence of every feature with respect to the target variable and includes only those features where this relationship could be significant.
- **RFE :** A method that recursively excludes the least important features according to their ranking in the model performance until it achieves the optimal subset of features.
- **VT :** Remove features with low variance that add less predictive power to the model.
- **Mutual Information :** It measures the amount of information each feature provides about the target variable and selects the most informative features.

we decided to take the 15 (can be tuned) most important features from each method. Then we applied a majority voting approach where we set a condition if a feature appeared a minimum three times then it will be selected.

D. Machine learning algorithms

We introduced several machine-learning classification algorithms including DecisionTree, ExtraTree, RandomForest, AdaBoost, Gradient Boost, and XGBoost individually; then we made different hybrid models like Stacking Classifier and two types of Voting Classifier. From deep learning architecture, we used CNN-LSTM and Federated Learning.

Stacking Classifier : This ensemble learning technique combines multiple baseline classifiers to form a strong predictive classifier. The predictions of baseline models work as an input feature for the meta-model. This technique averages their biases and variances to reduce the errors of individual models. We used Logistic Regression, Support Vector Machine, and DecisionTree as baseline algorithms.

Voting Classifier : This ensemble learning technique combines the predictions of the base models through a majority voting approach. we used DecisionTree, ExtraTree, and Logistic Regression as baseline models. Two types of voting are available here:

- **Hard Voting :**It is a straightforward method where the final prediction is the class label that receives the most votes from the baseline models.
- **Soft Voting :** Here the final prediction is the class label with the maximum average probability across all baseline models.

CNN-LSTM This architecture was formed using a convolution layer with 64 filters, a max pooling layer, two LSTM layers with 50 units, and a final dense layer for providing the label probability. The summary of the model shown in following table:

TABLE I
SUMMARY OF CNN-LSTM ARCHITECTURE

Layer (Type)	Output Shape	Param #
conv1d (Conv1D)	(None, 10, 64)	256
max_pooling1d (MaxPooling1D)	(None, 5, 64)	0
lstm (LSTM)	(None, 5, 50)	23,000
lstm_1 (LSTM)	(None, 50)	20,200
dense (Dense)	(None, 1)	51

Federated Learning : Federated Learning is a decentralized machine learning approach where two types of servers are present including a meta server and multiple client servers. Data is distributed to the client servers where each server hold different portion of the data. A model is initialized to the meta server and distributed to the local servers for training. The outcome of the model comes to the meta server and aggregation occurs. If the result is not satisfactory then the updated model again passes to the client server for learning

and this process continue until we get an optimal performance. This approach ensures data privacy and is effective for cyber security. We optimized our federated architecture for different number of clients. A neural network model was initialized to the meta server. The train set was again split for client servers. The model was trained for 30 epochs with batch size 32. In each local server, it was trained for 30 epochs for each client.

IV. Results and Discussion

This section presents the outcomes of the proposed methods for malware detection and analyzes their effectiveness. The performance of machine learning models, hybrid approaches, and Federated Learning is evaluated based on accuracy, precision, recall, and F1-score. Additionally, the results are discussed in the context of feature selection and privacy-preserving techniques, highlighting the practical implications of the findings. The dataset used for this study was sourced from Kaggle and contained no missing values or duplicate entries in any column. Additionally, no extreme outliers were present, ensuring that the dataset was clean and ready for analysis. Since the target variable was in categorical format, label encoding was applied, converting the malware class to ”0” and the benign class to ”1.” Afterward, five feature selection techniques were applied to identify the most relevant features. Using the majority voting approach, 12 key features were selected, including ’prio,’ ’vm-pgoff,’ ’free area cache,’ ’mm-users,’ ’total-vm,’ ’shared-vm,’ ’exec-vm,’ ’last-interval,’ ’nvcsw,’ ’maj-flt,’ ’lock,’ and ’end-data.’

The data was split into 75% for training and 25% for testing, with a random state of 42 ensuring reproducibility. Various machine learning classifiers, hybrid models, and neural network architectures were trained and tested, and their performances were evaluated based on standard metrics such as accuracy, recall, precision, and F1-score. The results are summarized in Table II. All the machine learning models demonstrated strong performance, with Random Forest and XGBoost achieving the highest accuracy of 100%. Both the Stacking Classifier and Voting Classifier (Soft and Hard Voting) also performed exceptionally well, achieving near-perfect metrics. The CNN-LSTM hybrid model achieved an accuracy of 99.92%, underscoring the effectiveness of neural network architectures in identifying malware patterns. The metrics of the hybrid models including Stacking Classifier, Soft Voting, and Hard Voting Classifier were almost equal as shown in TABLE II. Hence, all the models performed very well. From the neural network, CNN-LSTM hybrid architecture also show extraordinary performance with an accuracy score of 99.92%. Federated Learning, a decentralized approach that ensures data privacy, was evaluated using various configurations of client servers. The global model’s performance was measured for different numbers of clients, as shown in Table III. The architecture achieved a maximum accuracy of 99.99% with six clients, demonstrating the practicality of Federated Learning in scenarios where data privacy is critical.

The results highlight the advantages of using hybrid approaches and ensemble models for malware detection. Random Forest, XGBoost, and CNN-LSTM proved highly effective in identifying malicious patterns. The high performance of ensemble methods such as Stacking Classifier and Voting Classifier suggests that combining predictions from multiple models can significantly enhance detection accuracy. Federated Learning introduced an additional layer of privacy while maintaining competitive performance. This aspect is particularly relevant in cyber security applications, where sensitive user data must be protected. By keeping the data localized and aggregating only model updates, Federated Learning addresses key privacy concerns in malware detection. Moreover, the feature selection process played a crucial role in improving model performance by reducing dimensionality and retaining only the most significant features. The majority voting technique ensured that the selected features contributed effectively to the detection process.

In summary, this study demonstrates the potential of combining machine learning and deep learning architectures with Federated Learning to build robust and privacy-preserving malware detection systems. Future work could focus on expanding the dataset to include more diverse and recent malware samples, which would allow for even more comprehensive evaluation of the proposed methods.

V. Conclusion

This study demonstrates the importance of integrating advanced machine learning techniques and Federated Learning for robust Android malware detection. By leveraging a hybrid feature selection approach using majority voting, the study identified key features that significantly enhanced the performance of the detection models. Various machine learning and deep learning algorithms, including Random Forest, XGBoost, CNN-LSTM, and ensemble methods, were evaluated, showcasing exceptional accuracy and reliability in identifying malicious patterns. Federated Learning was introduced as a privacy-preserving solution, ensuring sensitive data remains localized while maintaining competitive performance. The decentralized nature of Federated Learning makes it particularly effective in cybersecurity applications, where data privacy is a critical concern. The results showed that this approach could achieve up to 99.99% accuracy while preserving data security.

The findings emphasize the advantages of hybrid approaches and ensemble learning techniques for malware detection, providing a scalable and adaptable framework for addressing emerging cybersecurity threats. Future work could focus on expanding the dataset to include more diverse and complex malware samples, further validating the proposed methods in real-world scenarios. Additionally, exploring other advanced techniques, such as transfer learning and reinforcement learning, could further improve the adaptability and efficiency of malware detection systems.

References

[1] Wikipedia Contributors, “Malware,” Wikipedia, Jul. 07, 2024. en.wikipedia.org/wiki/Malware

TABLE II
EVALUATION METRICS OF BASELINE MODELS

Classifiers	Accuracy	Recall	Precision	F1
DecisionTree	0.9999	0.9999	0.9998	0.9999
ExtraTree	0.9998	0.9998	0.9999	0.9998
RandomForest	1.0000	1.0000	1.0000	1.0000
AdaBoost	0.9757	0.9706	0.9806	0.9756
GradientBoost	0.9792	0.9597	0.9988	0.9788
XGBoost	1.0000	1.0000	1.0000	1.0000
Stacking	0.9999	0.9999	0.9998	0.9998
Hard Voting	0.9999	0.9998	1.0000	0.9999
Soft Voting	0.9999	0.9998	1.0000	0.9999
CNN-LSTM	0.9992	1.0000	0.9984	0.9992

TABLE III
METRICS OF FEDERATED LEARNING ARCHITECTURE FOR DIFFERENT NO. OF CLIENTS

No. of clients	Accuracy	Recall	Precision	F1
2	0.9986	0.9986	0.9986	0.9986
3	0.9990	0.9990	0.9990	0.9990
4	0.9997	0.9997	0.9997	0.9997
5	0.9988	0.9988	0.9988	0.9988
6	0.9999	0.9999	0.9999	0.9999

[2] "Malware Detection & Classification using Machine Learning," *IEEE Conference Publication*, IEEE Xplore. ieeexplore.ieee.org/document/9117547/

[3] S. Cook, "Malware attack statistics and facts for 2024: What you need to know," *Comparitech*, May 16, 2024. www.comparitech.com/antivirus/malware-statistics-facts/

[4] Trend Micro, "Android Malware Campaigns Reportedly Installed 250 Million Times," [Online]. Available: trendmicro.com/news/mobile-safety/android-malware-campaigns

[5] I. Shahadat, B. Bataineh, A. Hayajneh, and Z. A. Al-Sharif, "The use of machine learning techniques to advance the detection and classification of unknown malware," *Procedia Computer Science*, vol. 170, pp. 917–922, Jan. 2020. doi: 10.1016/j.procs.2020.03.110

[6] "Hybrid-based malware analysis for effective and efficiency Android malware detection," *IEEE Conference Publication*, IEEE Xplore. ieeexplore.ieee.org/document/9354315

[7] E. S. Alomari et al., "Malware detection using Deep Learning and Correlation-Based feature selection," *Symmetry*, vol. 15, no. 1, p. 123, Jan. 2023. doi: 10.3390/sym15010123

[8] I. Firdausi, C. Lim, A. Erwin, and A.S. Nugroho, "Analysis of machine learning techniques used in behavior-based malware detection," *Proceedings of the 2010 2nd International Conference on Advances in Computing, Control and Telecommunication Technologies (ACT 2010)*, pp. 201–203. doi: 10.1109/ACT.2010.33

[9] A. Yazdinejad, H. HaddadPajouh, A. Dehghantanha, R. Parizi, G. Srivastava, and M.-Y. Chen, "Cryptocurrency malware hunting: A deep recurrent neural network approach," *Applied Soft Computing*, vol. 96, 2020, article no. 106630. doi: 10.1016/j.asoc.2020.106630

[10] "Dynamic Permissions based Android Malware Detection using Machine Learning Techniques," *Proceedings of the 10th Innovations in Software Engineering Conference*, 2017. doi: 10.1145/3021460.3021485

[11] A. Kumar, K. Abhishek, K. Shah, D. Patel, Y. Jain, H. Chheda, and P. Nerurkar, "Malware Detection Using Machine Learning," *Communications in Computer and Information Science*, 2020, pp. 61–71. doi: 10.1007/978-3-030-65384-2_5

[12] M.S. Rana, C. Gudla, and A.H. Sung, "Evaluating machine learning models for android malware detection: A comparison study," *Proceedings of the 2018 VII International Conference on Network, Communication and Computing*, ACM, New York, NY, USA, pp. 17–21. doi: 10.1145/3301326.3301390

[13] Y. Ban, S. Lee, D. Song, H. Cho, and J.H. Yi, "FAM: Featuring Android Malware for Deep Learning-Based Familial Analysis," *IEEE Access*, vol. 10, 2022, pp. 20008–20018. doi: 10.1109/ACCESS.2022.3151357

[14] S.K. Smmarwar, G. Gupta, and S. Kumar, "A Hybrid Feature Selection Approach-Based Android Malware Detection Framework Using Machine Learning Techniques," in *Cyber Security, Privacy and Networking*, Springer, Berlin, Germany, 2022, pp. 347–356. doi: 10.1007/978-981-16-8664-1_30

[15] "Android Malware Dataset for Machine Learning," Kaggle. www.kaggle.com/datasets/shashwatwork/android-malware-dataset-for-machine-learning